Require: a design (its feature set and settings), an attacker profile a, a legitimate-traffic specification ℓ, a seed, and T attack minutes

derive three random streams from the seed: rng_ℓ (users), rng_w (attacker) and rng_p (whether a challenged person solves the challenge)

▷ *the same seed offers the same users to every scenario, so comparisons are paired; a digest of the offered trace is checked across designs*

build the pipeline of Algorithm 2 on the in-memory store, with fakes for CAPTCHA, address intelligence, HLR, the sender and WhatsApp; seed the returning population (20%) with verified history

for minute $= -\text{warm-up}, \ldots, T-1$ **do** ▷ warm-up 10 min, or 30 or 130 for pumping; the attack starts at minute 0

  $n_\ell \sim \mathrm{Poisson}(\ell.\mathrm{rate})$ from rng_ℓ and $n_a \sim \mathrm{Poisson}(a.\mathrm{rate})$ from rng_w; draw each arrival's time inside the minute and sort

  for each arrival at time t **do**

    apply every event due by t (receipt, code entry, resend) at its own time, then set the clock to t

    draw the request's exogenous quantities in advance: identity, address, number, CAPTCHA scores, delivery delay, whether and when the person would verify, WhatsApp reachability

    $r \leftarrow$ PROCESSREQUEST(request) ▷ Algorithm 2; a challenged user retries with probability 0.9, a challenge-buying attacker always

    if r sent an SMS **then** schedule the receipt at t plus the queue and delivery delays, and a code entry after the recipient's delay if it verifies

  ▷ *a pumper's carrier verifies as its profile says: always within 1 s, 60% after 30 s, or only when the block's statistic nears its threshold; a receipt faker schedules a failed receipt*

  apply events to the minute boundary; run the feedback loop's resolution timeouts (ONOUTCOME of Algorithm 2); run the adaptive job of Step 9 at its cadence

  ▷ *oracle job: told the legitimate rate; learned job: reads the pipeline's own counters, so cold start and poisoning are in play*

  record the attack requests offered and the SMS leaked this minute, and every user's progress through the funnel

drain: apply events until the receipt grace, the resolution timeout, the slowest code entry and a resend have elapsed after the last minute

return leakage per minute and containment; each user's funnel (dispatched, delivered, completed, challenged, refused) by cohort; verdict events; the attacker's verifications, challenges solved and bill; the configuration, seed, workload digest and code hash